

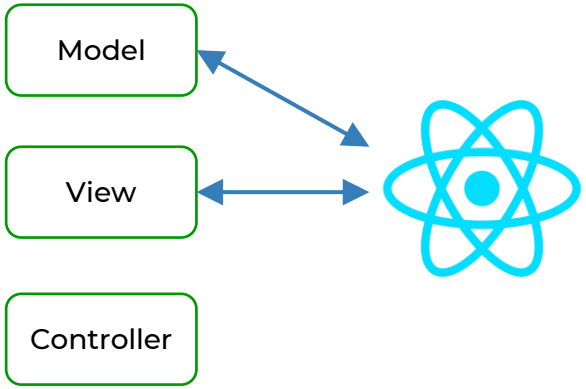
Class 03: React Framework

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

React

- React เป็น Java Script Library ซึ่งใช้ในการสร้าง UI
  - UI แต่ละชิ้น ถูกสร้างขึ้นมาเป็น component จากนั้น developer จึงประกอบ component ต่าง ๆ เข้าด้วยกันเป็น page, screen หรือ app เหมาะกับการทำ Component-based Application Development
  - ทำงานควบคู่กับ MVC ได้ดี โดย สร้าง control สำหรับ View ซึ่งทำงานคล้ายกับ controller ใน MVC model
  - ให้ top-level component ที่เก็บ state ของ data และส่งให้กับ component ที่อยู่ใต้ลงไป



React



- React components เป็น JavaScript functions รองรับการใช้งาน control structure, data structure ฯลฯ
- React component ถูก program ด้วย markup syntax เรียกว่า JSX
  - JSX เป็น JavaScript syntax extension
  - JSX เอื้อสำหรับ logic ในการ render ทำให้ง่ายต่อการ สร้าง อับเดต ลบ React component

- สำหรับ website เราสามารถใช้ React มากระดับใดก็ได้ โดยสามารถคงใช้ HTML DOM แบบเดิมควบคู่ไปกับการใช้งาน React ได้
  - บาง website ไม่ได้ต้องการ Single Page Web Application
  - ใช้ React component เฉพาะเพียงบางส่วน แล้วค่อยๆปรับใช้งานมากขึ้น
  - https://react.dev

React Syntax: JSX



JSX: JavaScript XML

- เป็น Syntax extension ของ JavaScript ที่มี ES6 เป็นพื้นฐาน
- JSX เป็น markup language ที่อนุญาตให้เขียน HTML ใน React โดยเปลี่ยน HTML tags ไปเป็น React elements
  - JSX ทำให้เราสามารถเขียน markup language เช่น เข้าไปใน code ได้
  - ใช้กับ React เพื่อระบุ UI ใช้ในการสร้าง elements ให้กับ React

Html + JavaScript ธรรมดา + React

```
const welcomeElem = React.createElement('h1', {}, Hello World!!');
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(welcomeElem);
```

JSX + React

```
const welcomeElem = <h1>Hello World!!</h1>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(welcomeElem);
```

React Syntax: JSX



JSX: JavaScript XML

- JSX เพื่อระบุ UI สร้าง elements ให้กับ React
  - สนับสนุน Tag ที่เป็น **Tag ของ components** จาก Library ที่ import เข้ามาใช้งาน นอกเหนือจาก HTML Tag ธรรมดา
    - ตัวอย่างการใช้งาน component จาก Next library

```
import Image from "next/image";
…
<Image src="/img/home.jpg“ alt="image"/>
```

    - ตัวอย่างการใช้งาน component จาก React Native library

```
import {Text, View} from 'react-native’;

…
<View><Text>Hello World!</Text></View>
```

React Syntax: JSX



- **JSX Expression**
  - JSX ทำงานกับ expression โดยคร่อม expression ด้วย { }
  - Expression อาจเป็น variable, property, หรือ JavaScript expression
  - JSX จะ execute expression และ return ผลลัพธ์ เพื่อนำผลลัพธ์จาก expression นั้น ๆ มาใช้งาน

จะถูกประมวลผลเป็น 20 ก่อนนำไปใช้งาน

```
const elem = <h1>Price is {10*2}</h1>;
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(elem);
```

- ตัวอย่างการใช้ { } เพื่อประมวลผล JavaScript Object ก่อนนำไปกำหนดเป็น property ของ component

```
<View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
  <Text>Hello, world!</Text>
</View>
```

JavaScript Object {flex:1, …} จะได้รับการประมวลผล

React Syntax: JSX



- **JSX สำหรับ HTML Block**
  - HTML Block ที่มีหลายบรรทัด ให้บรรจุไว้ในวงเล็บ ( )

```
const mainList = (
  <ol>
    <li>Honey</li>
    <li>Butter</li>
    <li>Sugar</li>
  </ol>
);
```

- **JSX Component**
  - ต้องบรรจุอยู่ใน Top-Level Element 1 element เท่านั้น

```
const heroSentence = (
 <p>Welcome to</p>
 <p>React Class</p>
);
```



```
const heroSentence = (
 <div>
  <p>Welcome to</p>
  <p>React Class</p>
 </div>
);
```

React Syntax: JSX



- JSX ต้องมีการปิด Element
  - Html element หรือ Tag ใน JSX ต้องปิด Tag ด้วย
  - Empty element ปิดด้วย />

```
import Image from "next/image";
…
<Image src="/img/home.jpg“ alt="image"/>
```

- Attribute `class` ใน JSX
  - Keyword `class` ของ HTML เป็น reserved word ใน JavaScript
  - เมื่อนำมาเขียนควบคู่กันใน JSX ให้ใช้เป็น attribute `className`

```
const heroSentence = (
 <div className="content1">
  <p>Welcome to</p>
  <p>React Class</p>
 </div>
);
```

React Components



- React components เป็น JavaScript functions ที่ return markup ซึ่งนำเสนอ component.
- เราสามารถใช้งาน component ซ้อนเข้าไปในอีก component หนึ่งได้ (nested component)

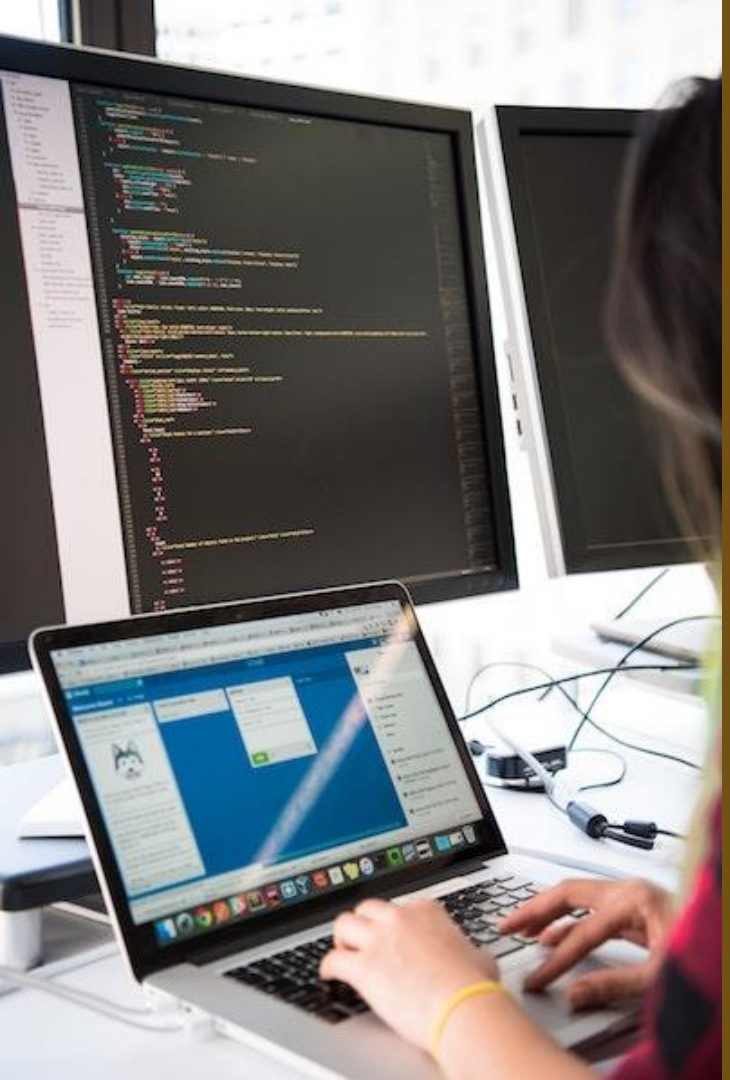
```
1  function MyButton() {
2    return (
3      <button>
4        I'm a button
5      </button>
6    );
7  }
8
9  export default function MyApp() {
10   return (
11     <div>
12       <h1>Welcome to my app</h1>
13       <MyButton />
14     </div>
15   );
16 }
```

Developing Full-Stack Web with React



- React เป็น library สำหรับประกอบ components เข้าด้วยกัน
- ในการพัฒนา Web Application แบบ Full Stack นั้น เราต้องการฟังก์ชั่นการทำงานอื่น ๆ ด้วย เช่น การดึงข้อมูล การทำ routing ฯลฯ
- Full-stack React framework ใช้พัฒนา web application ด้วย React โดย framework ที่นิยม ได้แก่ Next.js, Remix และ Gatsby

- Next.js เป็น front-end framework ที่ใช้ React ในการสร้าง user interfaces ด้วย UI components
  - React เป็น client-side technology ทำงานบน web browser
  - Next.js ใช้ Node.js (JavaScript runtime) จึงสามารถ execute JavaScript บน server-side (server-side rendering, SSR)
  - Next.js ทำให้สามารถ React บน server สร้างเว็บแบบ dynamic ทำการ render บน server เพื่อสร้างหน้า HTML หลีกเลี่ยงการทำงานบน client ตอนโหลดหน้าเว็บ

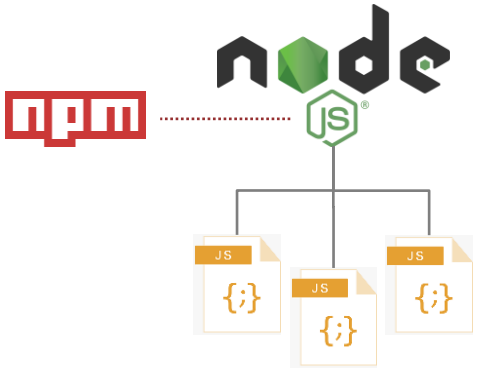


Workshop 3.1:
Creating React Web Application

Next.js Project

Node Project

- Class 03 ถึง Class 13 เป็น Frontend Programming ด้วย Next.js React Framework
  - ติดตั้งและบริหารจัดการ software package ต่าง ๆ รวมถึง build และ bundle ผ่าน npm (node package manager)





package.json

มี package.json เป็นไฟล์เก็บ metadata และ configuration เกี่ยวกับ project รวมถึง attributes ต่าง ๆ ที่ npm ใช้ในการติดตั้ง dependencies และ run scripts

Caution! Version Sensitive

- รูปแบบคำสั่งต่าง ๆ อาจใช้งานได้เฉพาะบางช่วง version ของ Software Package ที่ใช้งาน
- วิชา SW Dev Prac 2 ใช้งาน Next.js version 16 เป็นหลัก
- สังเกต version ของ package ได้จาก `package.json`
  - แต่ละ package มีเลข version กำกับ ในรูปแบบของ *X.Y.Z*
    - *X* = Major Release
    - *Y* = Minor Release
    - *Z* = Patch Release
  - สามารถมีสัญลักษณ์กำกับหน้าเลข version
    - *^version* หมายถึง Compatible with version ยอมรับ update ของ minor และ patch release เช่น ^1.2.5 ยอมรับ 1.2.5 อัปเดตได้ถึง <2.0.0
    - *~version* หมายถึง Approximately equivalent to version ยอมรับ update ของ patch release เช่น ~1.2.5 ยอมรับ 1.2.5 อัปเดตได้ถึง <1.3.0
    - *version* หมายถึง Exact version

package.json > ...

```
1  {
2    "name": "car-rental-app",
3    "version": "0.1.0",
4    "private": true,
     ▷Debug
5    "scripts": {
6      "dev": "next dev",
7      "build": "next build",
8      "start": "next start",
9      "lint": "eslint"
10   },
11   "dependencies": {
12     "next": "16.3.0",
13     "react": "19.2.8",
14     "react-dom": "19.2.8"
15   },
16   "devDependencies": {
17     "@types/node": "^20",
18     "@types/react": "^19",
19     "@types/react-dom": "^19",
20     "eslint": "^9",
21     "eslint-config-next": "16.3.0",
22     "typescript": "^5"
23   }
24 }
```

NPM for Node Project

npm command ที่ใช้งานบ่อย

- `npm install` : installs package and any packages that it depends on
  - ถ้า project มี `package-lock.json` อยู่ จะใช้ข้อมูลใน `package-lock.json` ติดตั้ง dependencies

- `npm install` *`package`* : install the specified package
  - โดยปกติ จะ save package นั้นลงใน `dependencies` โดย default

- `npm install -D` *`package`*
  - Save package นั้นลงใน `devDependencies`

- `npm install -E` *`package`*
  - ติดตั้ง exact version ของ package

- `npm prune` : removes "extraneous" packages
  - ลบ package ที่ไม่ปรากฏอยู่ใน dependencies ของ parent package

Creating React Web Application



- ติดตั้ง Node Version 20.9+ โดย Node มาพร้อมกับ Node Package Manager (npm)
  - Node.js LTS v20.16: https://nodejs.org/dist/v20.16.0/node-v20.16.0-x64.msi
  - npm version 5.2 ขึ้นไป มาพร้อมกับ npx (execute npm package binaries) ซึ่งทำให้สามารถติดตั้ง node package ต่างๆได้สะดวกมากยิ่งขึ้น
- เปิด Windows Power Shell

```
PS C:\Users\ongtang> node -v
v24.14.0
```

- สร้างไดเร็กทอรี่สำหรับเก็บ Web Application แล้ว cd เข้าไป
  - `cd D:\SWDevPrac2\ReactWeb\`
- ติดตั้ง `create-next-app` version 16.3.0
  - `npm install -g create-next-app@16.3.0`
- สร้าง React application
  - `npx create-next-app@16.3.0`

```
PS D:\SWDevPrac2\ReactWeb> npm install -g create-next-app@16.3.0

changed 1 package in 3s
PS D:\SWDevPrac2\ReactWeb> npx create-next-app@16.3.0
```

Creating React Web Application



- สร้าง Web Application ชื่อ car-rental-app ระบบจะถามการ configure application
  - What is your project named? car-rental-app
  - Would you like to use the recommended Next.js defaults? No, customize settings
- Next.js มาพร้อม TypeScript, ESLint และ Tailwind CSS configuration
  - Would you like to use TypeScript? No / Yes → เลือกใช้ TypeScript
  - Which linter would you like to use? ESLint / Yes → ใช้ ESLint เป็น Linter (default)
  - Would you like to use React Compiler? No / Yes → ยังไม่ใช้ (default)
  - Would you like to use Tailwind CSS? No / Yes → ยังไม่ใช้ (default)
- เราสามารถเลือกให้จัดเป็น project file ไว้ใน src directory ได้ และใช้ App Router (default)
  - Would you like to use `src/` directory with this project? No / Yes
  - Would you like to use App Router (recommended)? No / Yes
  - Would you like to customize the default import alias? No / Yes
  - Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? No / Yes

Creating React Web Application

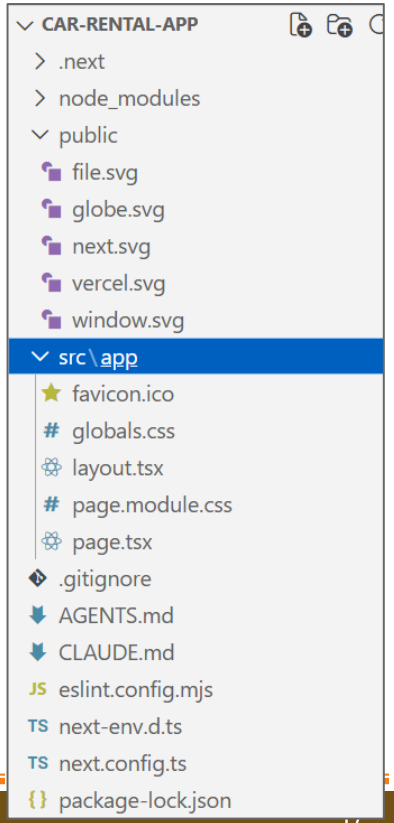
- เมื่อสร้าง project เสร็จ จะได้โฟล์เดอร์ `car-rental-app` ภายในมีไฟล์ project ต่าง ๆ
- cd เข้าไปใน project directory
  - `cd car-rental-app`
- โฟล์เดอร์ `car-rental-app` เปิด ด้วย VS Code
- ที่ Terminal ของ VS Code พิมพ์คำสั่ง Start React Application บน localhost
  - `npm run dev`

```
PS D:\SWDevPrac2\ReactWeb> cd .\car-rental-app\
PS D:\SWDevPrac2\ReactWeb\car-rental-app> npm run dev
```

- React Application จะทำงานที่ http://localhost:3000/

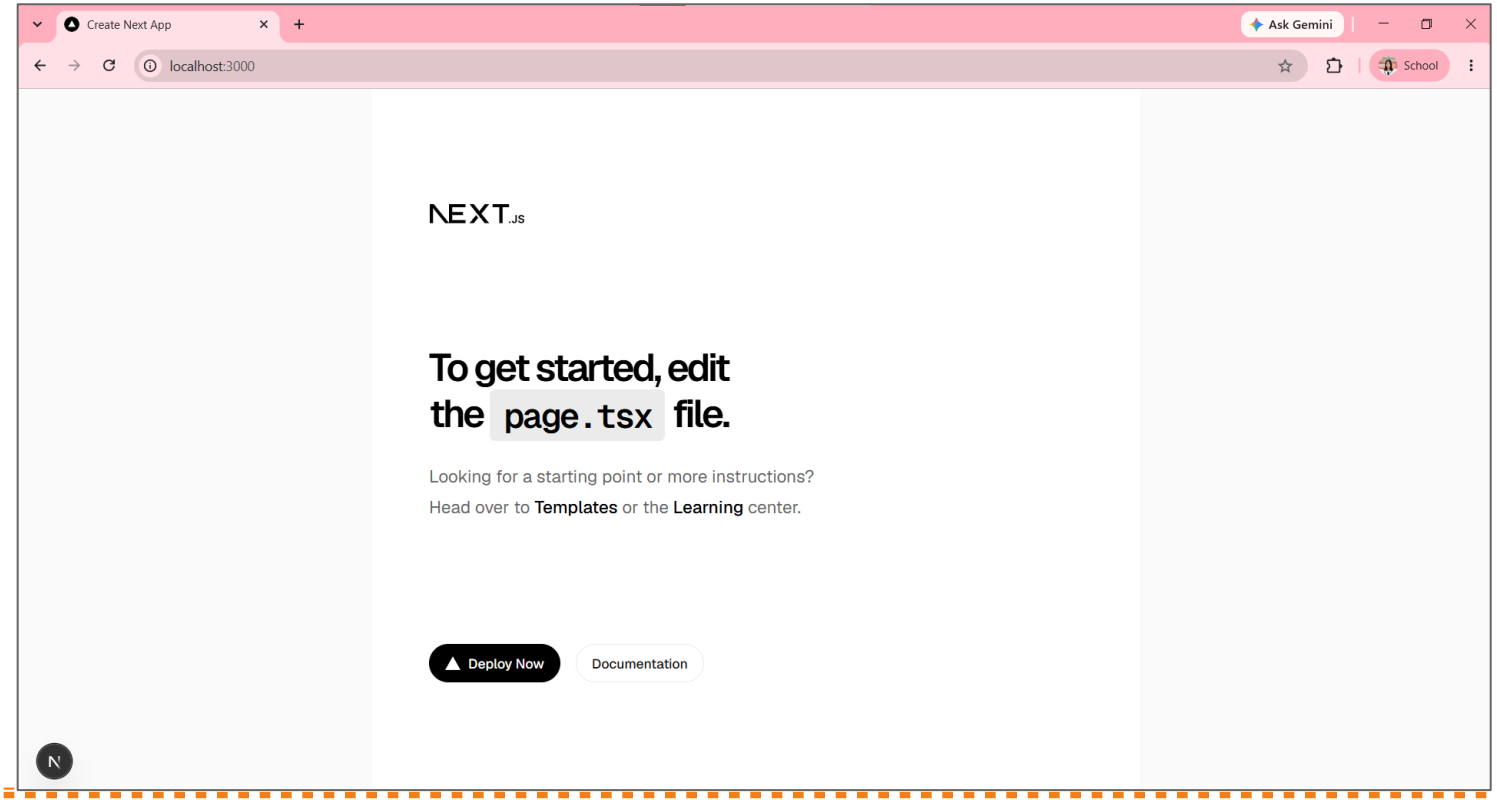
หมายเหตุ : หากต้องการ build เพื่อนำไป run บน production server ให้สร้าง bundle ด้วยคำสั่ง

- `npm run build`

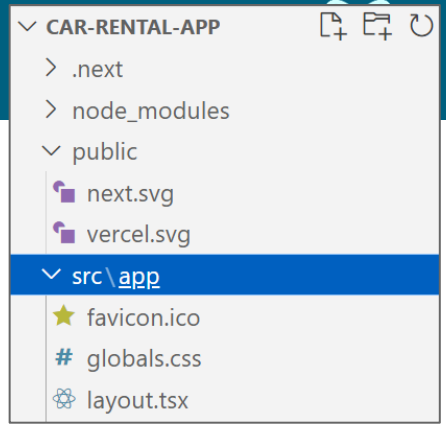


Creating React Web Application

- เปิด Web Browser ไปที่ http://localhost:3000/

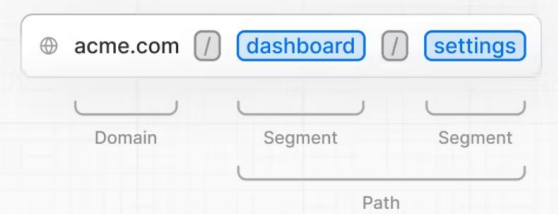


Next.js Basic: Project Structure

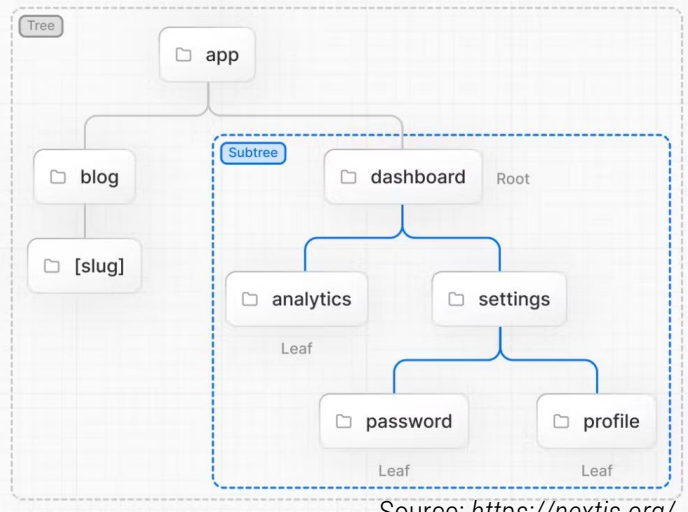


- โฟล์เดอร์ `public` : เก็บ Static Assets เช่น ไฟล์ หรือ ไฟล์รูปต่าง ๆ
  - เรียกใช้งานผ่าน base URL (`/`) ของ web เช่น สำหรับ image ชื่อ `vercel.svg` ในโฟล์เดอร์ `public` หากจะกำหนดให้เป็น source ของ image element อ้างอิงเป็น `src="/vercel.svg"`
  - เฉพาะ assets ที่บรรจุอยู่ในโฟล์เดอร์ `public` ช่วง build time เท่านั้น ที่สามารถเรียกใช้งานได้

- โฟล์เดอร์ `src\app` : สำหรับ App Router
  - ไดเร็กทอรี่ `app` เป็นจุดที่ App Router ทำงานสำหรับ Next.js โดยทำหน้าที่เป็น root สำหรับการ navigate



Source: *https://nextjs.org/*



Source: *https://nextjs.org/*

Next.js Basic: Project Structure



- `next.config.ts` : เป็นไฟล์ TypeScript ที่ Root ของ project directory สำหรับเก็บ configuration ของ Next.js

สร้าง option ที่มี property และ value ซึ่งระบุ configuration ของ Next.js

Export ออกไปใช้งาน

```
1  /** @type {import('next').NextConfig} */
2  const nextConfig = {};
3
4  export default nextConfig;
```

ตัวอย่าง option

- `basePath` กำหนด path prefix สำหรับ web app ที่พัฒนา ใช้เมื่อ web app นั้นจะนำไปไว้ที่ sub-path ของ domain
  - เมื่อ app ถูก build จะทำให้ Next.js นำ base path ที่ระบุไปเติมหน้า link หรือ router ของ app

```
basePath: '/docs',
```

- `httpAgentOptions` ตั้งค่า HTTP Agent Options

```
httpAgentOptions: {
  keepAlive: false,
},
```

- เพิ่มเติม https://nextjs.org/docs/app/api-reference/next-config-js

Next.js Basic: Project Structure



- `next-env.d.ts` : เป็นไฟล์ TypeScript ที่ Root ของ project directory
  - สำหรับให้ TypeScript compiler รู้จัก types ต่าง ๆ ของ Next.js
  - ไม่ควรลบ หรือแก้ไข โดยไฟล์นี้จะถูก regenerate ในทุกการ rebuild

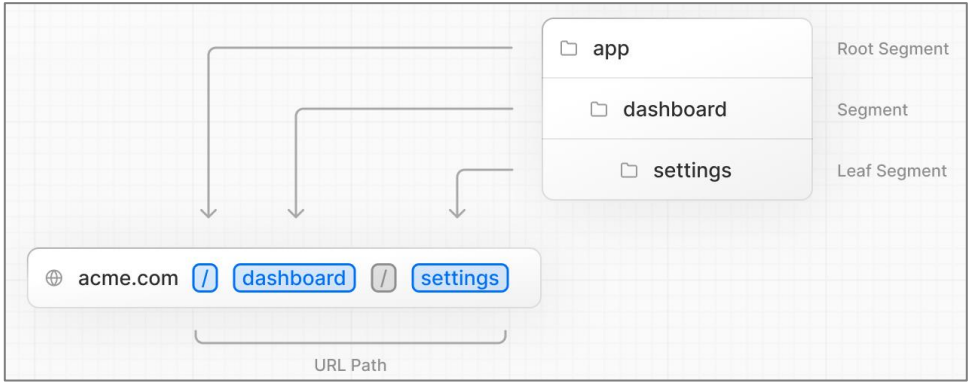
- `tsconfig.json` : สำหรับกำหนด configuration ให้ TypeScript โดยวางไฟล์ไว้ใน Root Directory ของ TypeScript
  - Compiler Options
  - ไฟล์ที่จะรวม (include) types เข้ามาใช้งาน
  - ไฟล์ที่จะไม่รวม (exclude) types ออกจากการใช้งาน
  - นิยาม path scope ต่าง ๆ

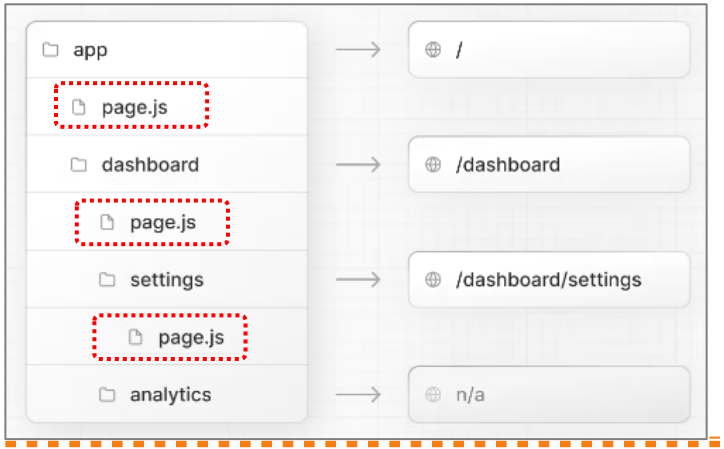
tsconfig.json > ...

```jsonc
{
  "compilerOptions": {
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
```

Next.js Basic: Pages & Layouts

- Next.js ใช้ file-system based router โดยใช้ folder ในการระบุ route ต่าง ๆ
  - แต่ละ folder นำเสนอ route segment ที่ map เข้ากับ URL segment
  - สร้าง nested route ผ่าน nest folders
  - ไฟล์ `page.js` หรือ `page.jsx` หรือ `page.tsx` เป็นไฟล์พิเศษที่ทำให้ route segments นั้น ๆ มองเห็นเป็นสาธารณะ

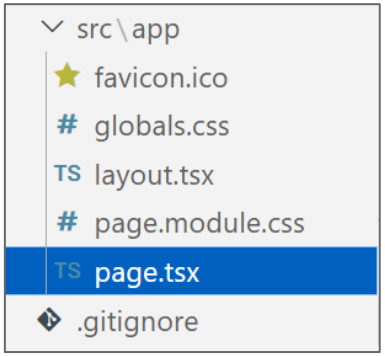




Next.js Basic: Pages & Layouts



- **Page**
  - page เป็น UI ที่ unique สำหรับ route
  - ระบุ page โดยการ export component จากไฟล์ `page.js` หรือ `page.jsx` หรือ `page.tsx`



- **Layout**
  - Layout เป็น UI ที่ share ระหว่างหลาย page
  - ในระหว่างการ navigate นั้น layout จัดเก็บ state และไม่ได้รับการ re-render
  - ระบุ layout โดย default ด้วยการ export React component จากไฟล์ `layout.js` หรือ `layout.jsx` หรือ `layout.tsx`

Next.js Basic: Pages & Layouts



- **Layout**
  - Component รับ children prop ที่จะถูก populate โดย child layout (ถ้ามี หรือ child page ระหว่างการ render)
  - จำเป็นต้องมี Root layout ที่ระดับ top level ของ app directory และ apply กับ route ทั้งหมด

layout.tsx

```tsx
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en">
      <body className={`${geistSans.variable} ${geistMono.variable}`}>
        {children}
      </body>
    </html>
  );
}
```

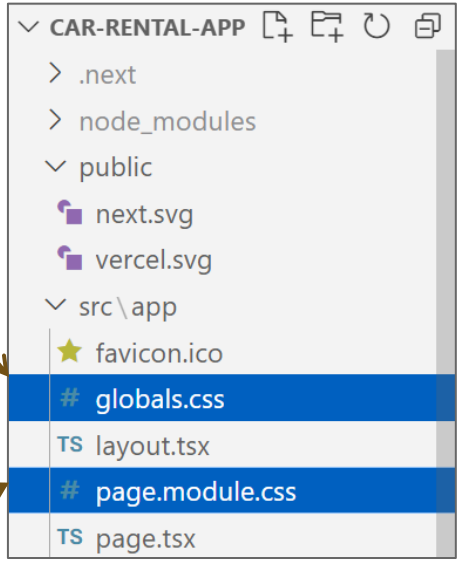
Next.js Basic: Styling

- Next.js รองรับ styling ในหลากหลายรูปแบบ ตัวอย่างเช่น
- Global CSS
  - Next.js มาพร้อม `app/globals.css` สามารถ import Global style ไปใช้ใน layout, page หรือ component ใน app directory ได้
  - ใช้งานในลักษณะเดียวกับ CSS ดั้งเดิม แต่หากใส่ทุกอย่างไว้ใน global css จะยากต่อการจัดการ

layout.tsx

```tsx
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";
```

- CSS Modules
  - Develop สามารถสร้าง CSS ที่มี scope ในระดับ local ได้
  - ป้องกันการขัดแย้งกันหากใช้ชื่อซ้ำกัน (naming conflict)
  - ลงท้ายด้วย `.module.css`
  - สามารถ import ไปใช้ในไฟล์อื่น ๆ ได้



Next.js Basic: Styling



- **Inline Styling**
  - สามารถใส่ inline style attribute เข้าไปใน markup ได้ โดยค่าของ attribute ต้องเป็น JavaScript object

```
<h1 style={{color: "red"}}>Hello Style!</h1>
```

  - inline CSS ต้องเขียนเป็น JavaScript object ดังนั้น properties ที่มี 2 คำ เช่น `background-color` ต้องเขียนเป็น camel case เช่น `backgroundColor`

```
<h1 style={{backgroundColor: "lightblue"}}>Hello Style!</h1>
```

Next.js Basic: Image



- Next.js มาพร้อม `Image` component ซึ่งต่อยอดจาก HTML `<img>` element โดยเพิ่ม features สำหรับการ optimize image โดยอัตโนมัติ
  - **Size Optimization**: ใช้งาน image ที่มีขนาดเหมาะสมสำหรับอุปกรณ์ให้อัตโนมัติ
  - **Visual Stability**: ให้ความเสถียรในการแสดงผล โดยป้องกัน layout shift ขณะโหลดรูปภาพ
  - **Faster Page Loads**: โหลดรูปภาพเฉพาะเมื่อภาพจะอยู่ใน viewport โดยใน lazy loading ของ Web Browser และกำหนดให้มีการ blur ส่วน placeholder ขณะโหลดได้
  - **Asset Flexibility**: On-demand image resizing สำหรับภาพที่จัดเก็บบน remote servers
- การใช้งาน
  - import component

```
import Image from 'next/image'
```

  - ระบุ local source

```
<Image src="img/cover.png" alt="Welcome" width={300} height={300}
placeholder="blue"/>
```

Next.js Basic: Image



- การใช้งาน
  - ระบุ remote source
    - ระบุ width และ height เนื่องจาก Next.js ไม่สามารถคำนวณให้ตอน build time ได้

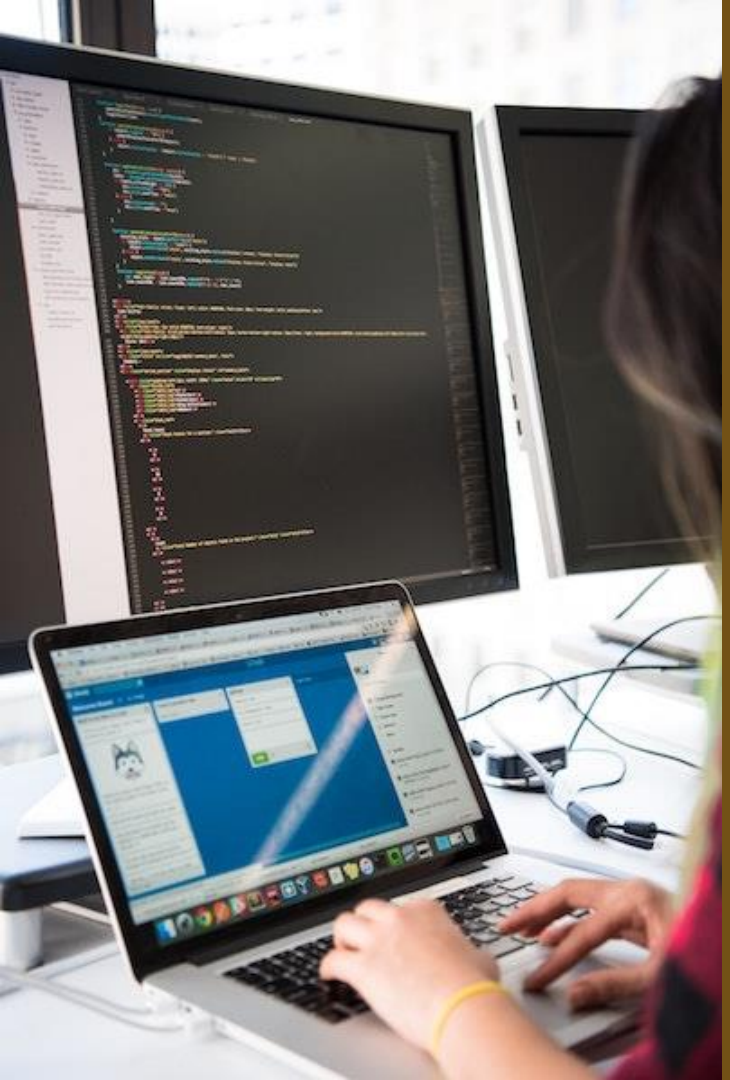
```
<Image src="https://mypicrepo.com/me.png" alt="My Picture"
width={500} height={500} />
```

  - property `fill` สำหรับกำหนดขนาดให้ image นั้นขยายเต็ม parent element หรือไม่

```
<Image src="img/cover.png" alt="Welcome Picture" fill={true} />
```

  - property `priority` สำหรับ image ที่เป็นองค์ประกอบหลักของ Viewport (Largest Contentful Paint (LCP) element) เพื่อเป็นข้อมูลให้ Next.js ใช้ในการ load ภาพ

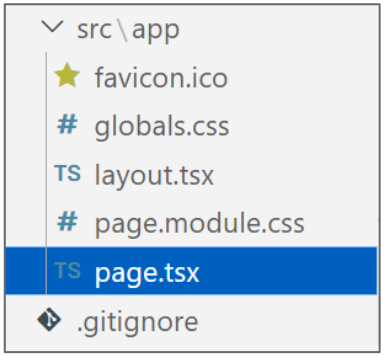
```
<Image src="img/cover.png" alt="Welcome Picture" priority fill={true} />
```



Workshop 3.2:
Working with React Component

Working with React Component

- เปิด Visual Studio Code แล้วเปิดโฟล์เดอร์
  - `car-rental-app`
- ปรับแต่งโปรเจกต์
  - `src/app/page.tsx` เป็นหน้าเริ่มการทำงาน
- จากไฟล์ `page.tsx` จะเห็นได้ว่า
  - มีการ export function `home()` ออกไปใช้งาน
  - function `home()` return component ที่จะถูก render ไปแสดงผลบนหน้าจอ




```tsx
export default function Home() {
  return (
    <div className={styles.page}>
      <main className={styles.main}>
        <Image
          className={styles.logo}
          src="/next.svg"
          alt="Next.js logo"
          width={180}
          height={38}
          priority
        />
        <ol>
          <li>
            Get started by editing <code
```

HTML `main` tag แสดงส่วน content หลักของ HTML document

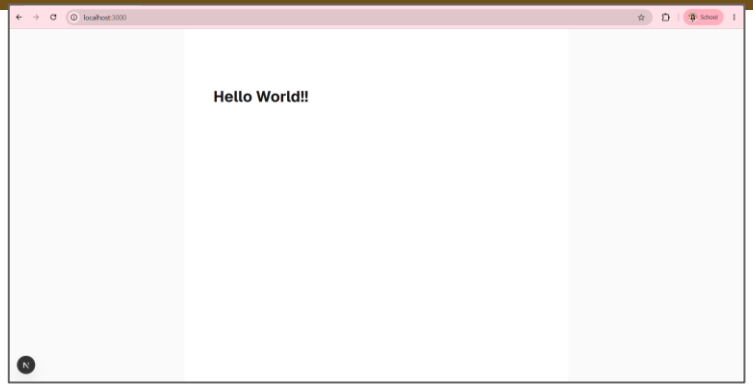
Working with React Component



- เปลี่ยนเนื้อหาไฟล์ page.tsx หน้า Web จะ update อัตโนมัติ

```tsx
import Image from "next/image";
import styles from "./page.module.css";

export default function Home() {
  return (
    <div className={styles.page}>
      <main className={styles.main}>
      <div>
        <h1>Hello World!!</h1>
      </div>
      </main>
    </div>
  );
}
```



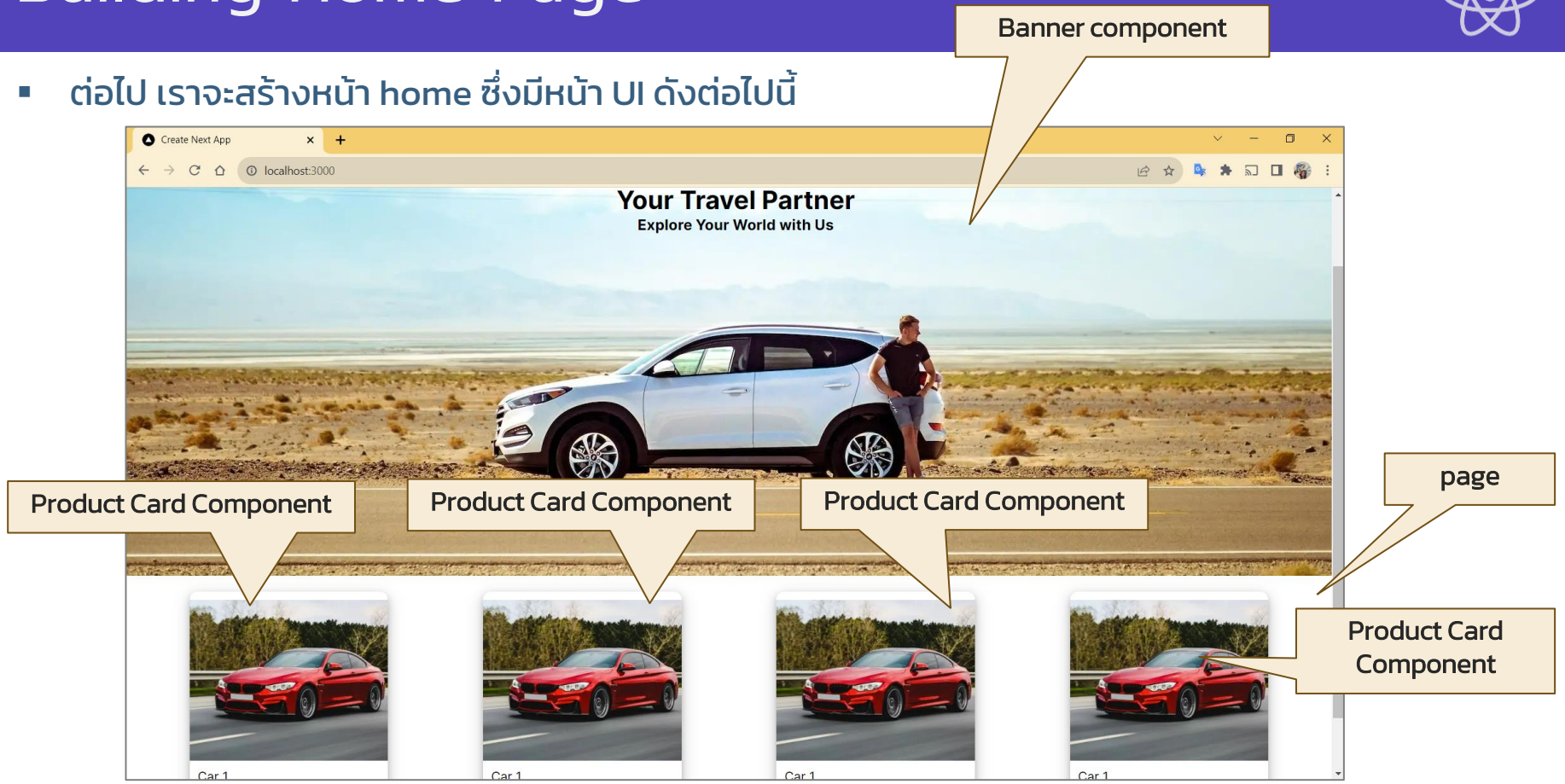


- ปรับ page.module.css
  - ให้ เนื้อหาแสดงเต็มหน้า และอยู่กลางหน้า

```css
.main {
  display: flex;
  flex: 1;
  width: 100%;
  flex-direction: column;
  align-items: flex-start;
  justify-content: flex-start;
  background-color: var(--foreground);
}
```

Building Home Page

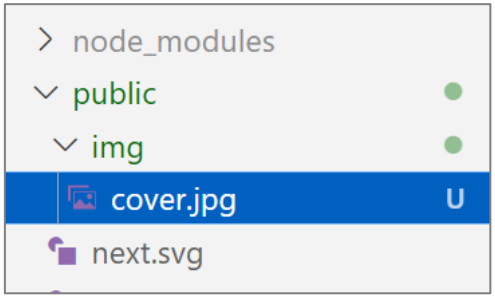
- ต่อไป เราจะสร้างหน้า home ซึ่งมีหน้า UI ดังต่อไปนี้



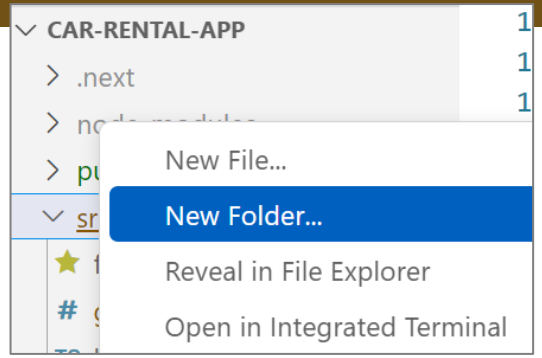
Building Home Page

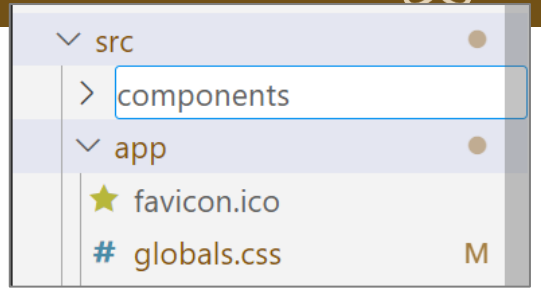


- Next.js จัดเก็บ public assets ไว้ที่โฟลเดอร์ `public`
  - โฟลเดอร์นี้เข้าถึงได้จาก page ต่าง ๆ หลังการ render
  - สร้างโฟล์เดอร์ `img` เพื่อเก็บรูปภาพ
  - วางไฟล์ `cover.jpg` ลงไปเก็บ เพื่อใช้เป็น background ของส่วน banner ด้านบน



Building Home Page





สร้าง Banner Component

- สร้างโฟล์เดอร์ components ใน โฟล์เดอร์ src

- ในโฟล์เดอร์ components สร้างไฟล์ Banner.tsx และ banner.module.css

- ไฟล์ banner.module.css
  - สร้าง Style ให้กับ div ที่เป็น container ของกล่อง banner

```
.banner {
    display: block;
    padding: 5px;
    margin: 0;
    width: 100vw;
    height: 80vh;
    position:relative;
}
```

Building Home Page

- ไฟล์ Banner.tsx
  - สร้าง Banner component ในรูปแบบ function พร้อม export ออกไป

src > components > TS Banner.tsx > ...

```tsx
import styles from './banner.module.css'

export default function Banner () {
    return (
        <div className={styles.banner}>

        </div>
    );
}
```

- แก้ไขไฟล์ page.tsx
  - ให้ main tag บรรจุ Banner component

```tsx
import Image from "next/image";
import styles from "./page.module.css";
import Banner from "@/components/Banner";

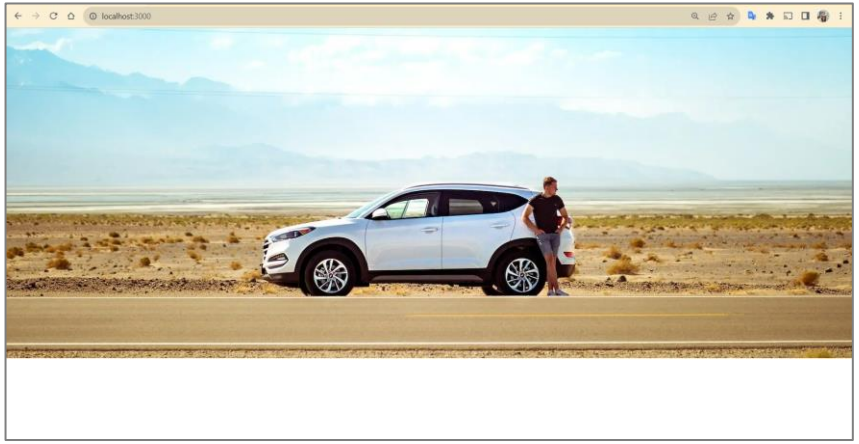
export default function Home() {
  return (
    <div className={styles.page}>
      <main className={styles.main}>
        <Banner/>
      </main>
    </div>
  );
}
```

Building Home Page

- ไฟล์ Banner.tsx
  - เพิ่ม Image ให้กับ Banner

```
import styles from './banner.module.css'
import Image from 'next/image';

export default function Banner () {
    return (
        <div className={styles.banner}>
            <Image src={'/img/cover.jpg'}
            alt='cover'
            fill={true}
            objectFit='cover'/>
        </div>
    );
}
```



Building Home Page

- ไฟล์ banner.module.css
  - สร้าง Style ให้กับ div ที่เป็น container ของ Text บน banner
- ไฟล์ Banner.tsx
  - เพิ่ม Text ให้กับ Banner

```
.bannerText {
    position:relative;
    top: 100px;
    z-index: 20;
    text-align: center;
}
```

```
export default function Banner () {
    return (
        <div className={styles.banner}>
            <Image src={'/img/cover.jpg'}
            alt='cover'
            fill={true}
            objectFit='cover'/>
            <div className={styles.bannerText}>
                <h1>Your Travel Partner</h1>
                <h3>Explore Your World with Us</h3>
            </div>
        </div>
    );
}
```

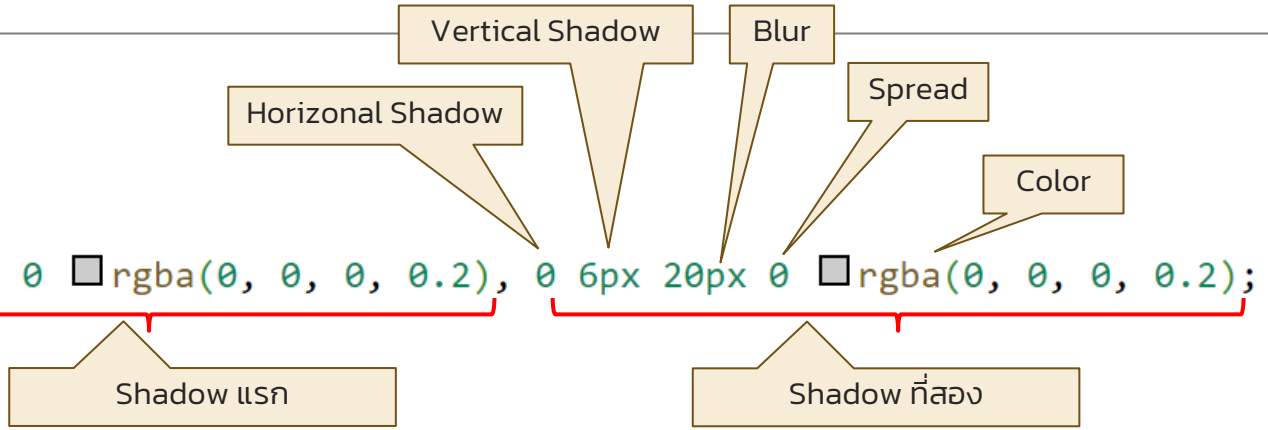


Building Home Page

สร้าง Product Card Component

- ในโฟล์เดอร์ components สร้างไฟล์ ProductCard.tsx และ productcard.module.css
- ไฟล์ productcard.module.css
  - สร้าง Style ให้กับ div ที่เป็น card container

```
div.card {
    width: 250px;
    height: 300px;
    background: white;
    box-shadow: 0 4px 8px 0 rgba(0, 0, 0, 0.2), 0 6px 20px 0 rgba(0, 0, 0, 0.2);
    border-radius: 10px;
    padding-top: 10px;
}
```



Building Home Page



สร้าง Product Card Component

- ไฟล์ `productcard.module.css`
  - สร้าง Style ให้กับ div ที่บรรจุ image

```
div.cardimg {
    width:100%;
    height:70%;
    position:relative;
}
```

Building Home Page



- ไฟล์ ProductCard.tsx
  - สร้าง ProductCard component ในรูปแบบ function พร้อม export ออกไป

```tsx
import styles from './productcard.module.css'
import Image from 'next/image';

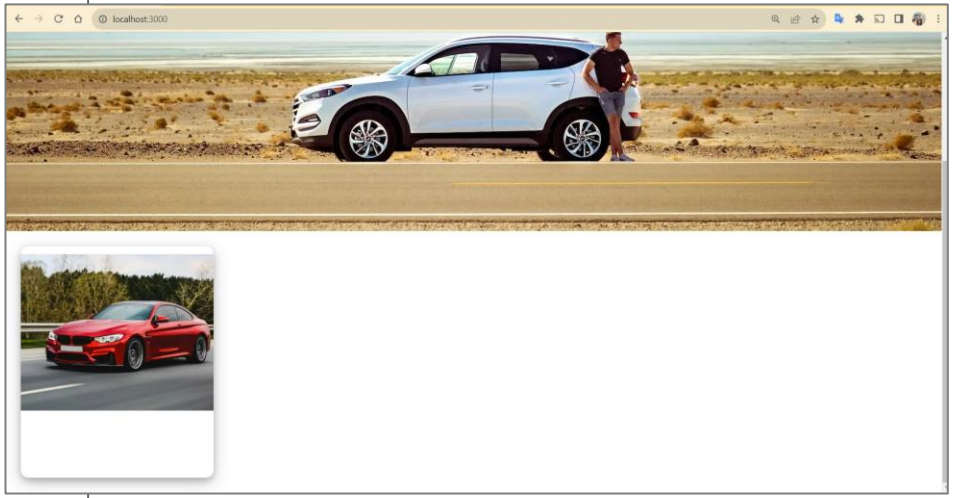
export default function ProductCard () {
    return (
        <div className={styles.card}>
            <div className={styles.cardimg}>
            <Image src={'/img/car1.jpg'}
                alt='Product Picture'
                fill={true}
                objectFit='cover'/>
            </div>
        </div>
    );
}
```

Building Home Page

- แก้ไขไฟล์ page.tsx
  - ให้ main tag บรรจุ ProductCard component

```tsx
import Image from "next/image";
import styles from "./page.module.css";
import Banner from "@/components/Banner";
import ProductCard from "@/components/ProductCard";

export default function Home() {
  return (
    <div className={styles.page}>
      <main className={styles.main}>
        <Banner/>
        <div style={{margin:"20px"}}>
        <ProductCard/>
        </div>
        </main>
    </div>
  );
}
```

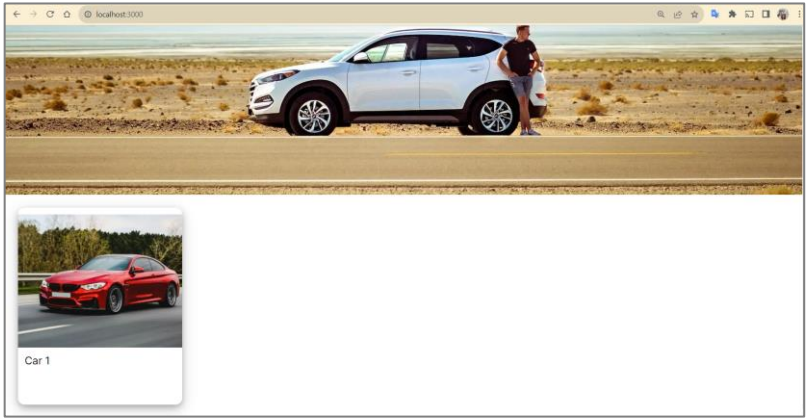


Building Home Page

- ไฟล์ productcard.module.css
  - สร้าง Style ให้กับ div ที่เป็น card text
- ไฟล์ ProductCard.tsx
  - เพิ่ม div ที่เป็น cardtext ให้กับ ProductCard component

```
div.cardtext {
    padding: 10px;
    height: 30%;
}
```

```
export default function ProductCard () {
    return (
        <div className={styles.card}>
            <div className={styles.cardimg}>
            <Image src={'/img/car1.jpg'}
                alt='Product Picture'
                fill={true}
                objectFit='cover'/>
            </div>
            <div className={styles.cardtext}>Car 1</div>
        </div>
    );
}
```



Building Home Page

- แก้ไขไฟล์ page.tsx
  - เพิ่ม ProductCard component ลงไปหลาย ๆ อัน

```
6   export default function Home() {
7     return (
8       <div className={styles.page}>
9         <main className={styles.main}>
10          <Banner/>
11          <div style={{margin:"20px", width: "100%", display: "flex", flexDirection: "row",
            alignContent:"space-around", justifyContent:"space-around", flexWrap:"wrap"}}>
12          <ProductCard/>
13          <ProductCard/>
14          <ProductCard/>
15          <ProductCard/>
16        </div>
17        </main>
18      </div>
19    );
20  }
```

